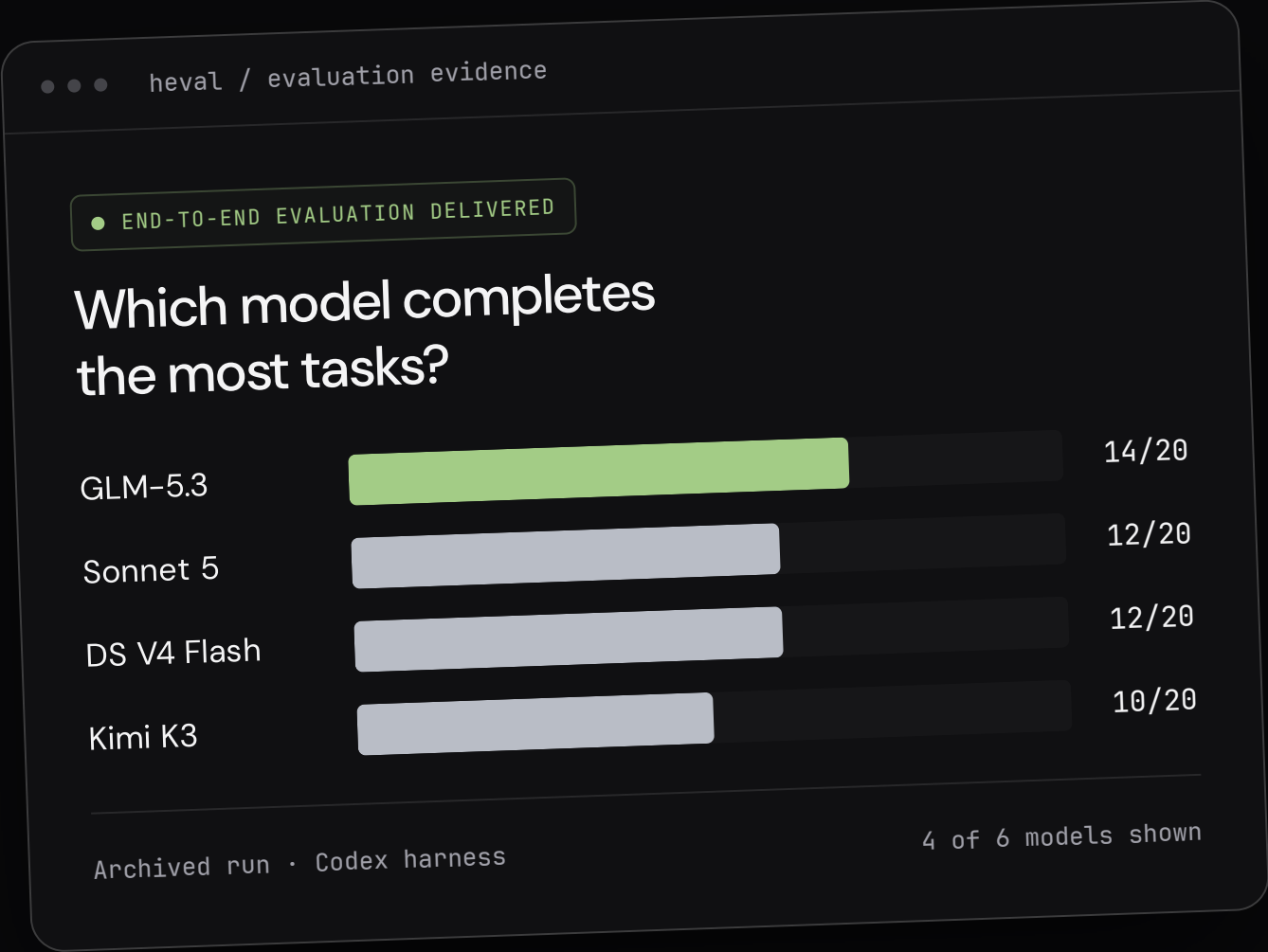


SOFTWARE ENGINEERING CAPSTONE STUDIO

Better agent decisions. Backed by evidence.

From completed evaluations and published insights to a CLI and cloud platform others can use.

Matthew Feroz
SSW695 · Fall 2026 · Wednesday evenings
Prof. Rich Kempinski



FROM AN AUTHOR’S WORKFLOW TO A USABLE PRODUCT

Let other developers run the same journey.

The evaluation and publishing workflow exists. The next step is making it accessible through an installed CLI and a cloud workspace.

USER 01 / DEVELOPERS

Start from the terminal

Install a package, configure a comparison, and launch it without cloning Heval’s source.

A repeatable CLI workflow that fits how developers already work.

USER 02 / EVALUATORS

Run beyond the laptop

Submit work to a cloud worker and follow progress from a browser.

Access execution and results across devices, even after closing the original session.

USER 03 / TEAMS

Review & publish together

Open saved evaluations, inspect evidence, and create charts in the hosted application.

Shared reports and exports without each person operating a local server.

A PLATFORM FOR THE FULL EVALUATION LIFECYCLE

Evaluate anywhere. Build on every result.

Make Heval the place developers and teams use to run, compare, reproduce, and publish coding-agent evaluations—from the terminal to the cloud.

One workflow, many environments

A shared evaluation plan across local machines, cloud workers, and future compute providers.

Continuous evaluation

Scheduled runs, version comparisons, regression alerts, and CI integration.

A collaborative evidence base

Team workspaces, reusable benchmark collections, run history, and reviewable findings.

This semester: release the CLI-to-cloud workflow and make the complete application available online.

COMPLETED WORK IS THE FOUNDATION

Already evaluated. Already published.

• END-TO-END WORKFLOW COMPLETED

6

models compared

20

tasks per model

120

completed trials

Run evaluations → Analyze results → Publish findings

Heval produced the completion, cost, and timing comparisons used in the published Merge post.

Existing CLI: result viewing and setup checks

Existing web foundation: Studio, hosted reports, and connected-runner previews

Next: turn that proven workflow into an installable, cloud-connected product.

Merge @merge_api

We tested 5 open-weight models against @AnthropicAI Claude Sonnet 5 on 20 real coding tasks.

- @Zai_org GLM 5.3
- @Zai_org GLM 5.3 Flash
- @deepseek_ai DeepSeek V4 Pro
- @deepseek_ai DeepSeek V4 Flash
- @Kimi_Moonshot Kimi K3

GLM 5.3 won at a 1/10 of Claude's cost:

5 open-weight models vs 1 frontier model

Completion rate
↑ Higher is better

Cost per success
↓ Lower is better

Median time
↓ Lower is better

9:30 AM · Sep 15, 2026 · 27.5K Views

42

35

340

85

Published Merge post · 27.5K views at capture

THE NEXT PRODUCT RELEASE

Expand the CLI. Deliver the cloud application.

PRIORITY	MAJOR FEATURE	SEMESTER OUTCOME
P0	Expanded CLI Build on open, doctor, and runner work	Authenticate, submit a run, inspect status, and cancel
P0	Complete cloud application Promote the existing web foundation	Hosted projects, Studio, reports, and image/thread exports
P0	Remote evaluation execution One supported cloud worker environment	Queue, progress, recovery, cancellation, and saved results
P0	CLI ↔ cloud result sync Local artifacts become account-owned reports	Upload results, reopen online, and share a published report
P1	Team & comparison workflows Extend existing report collaboration	Invitations, run history, and baseline comparison

P0 = core semester delivery · P1 = next priority
Testing, documentation, access control, and release setup are part of the core.

CONCRETE RELEASE TARGET

Terminal → cloud → report

A new user installs the CLI, runs on a supported cloud worker, and edits and exports the saved result online.

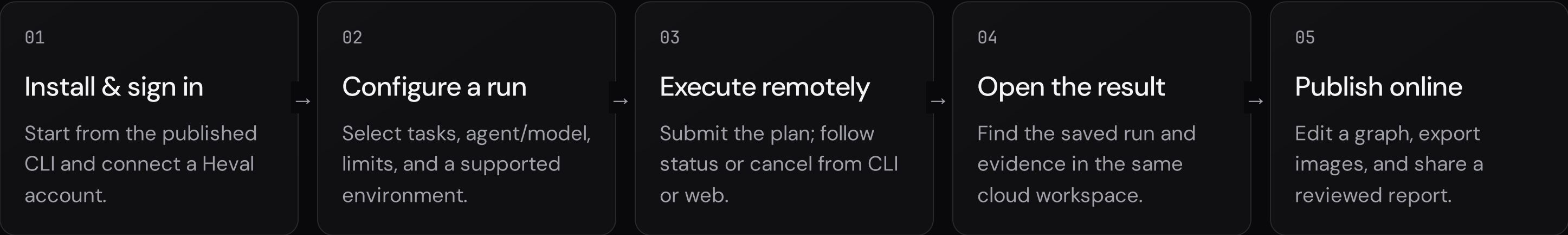
Next expansion

Multiple compute providers, scheduled regression sweeps, CI integrations, and organization administration.

THE TARGET PRODUCT EXPERIENCE

“Run my evaluation. Send me the report.”

Install once, use a shared evaluation plan, and continue in the browser wherever you are.



Proposed CLI interaction

```
heval login
heval run ./eval.json --target cloud
heval push ./results
```

Design examples for the next release; these commands are not yet shipped.

The analysis engine is already working

Today’s local demo shows the editor and publishing flow that the cloud product will expose.

Open existing Studio ↗

REQUIREMENTS FOR THE NEXT RELEASE

Three epics connect the terminal to the cloud.

EPIC · multiple sprints → FEATURE · capability → STORY · one sprint

E1 / DEVELOPER CLI

An installable workflow

F1.1 · Run submission

s1 Submit a validated plan and receive a run ID.

F1.2 · Account & artifacts

s4 Upload a local result into my signed-in workspace.

E2 / CLOUD APPLICATION

A complete online workspace

F2.1 · Hosted Studio & exports

s2 Export a saved report entirely from the hosted app.

F2.2 · Persistent reports

s5 Reopen my saved report from a second device.

E3 / REMOTE EXECUTION

Durable evaluation runs

F3.1 · Worker lifecycle

s3 Reconnect to a run without creating another execution.

F3.2 · Results & cancellation

s6 Cancel a run and preserve the partial evidence.

Each story becomes an owned GitHub issue with acceptance criteria and a sprint; existing behavior forms the baseline.

SPRINT-SIZED SLICES OF THE CLI + CLOUD RELEASE

Test the new product capabilities.

S1 · E1 / F1.1

Submit from the CLI

As a developer, I want to submit a reviewed plan so I can start a remote evaluation.

Given an authenticated CLI and a valid plan...

When I submit to a supported cloud worker...

Then create one queued run, return its ID and web URL, and show the same plan online.

S2 · E2 / F2.1

Export from the cloud

As a researcher, I want to export a hosted report without running a local server.

Given my saved report and no local Heval process...

When I select a question and export its graph...

Then download a PNG matching the hosted preview; another account cannot access the private source.

S3 · E3 / F3.1

Reconnect to an active run

As an evaluator, I want to recover a connection without starting the work again.

Given a running cloud job and a network interruption...

When the worker and client reconnect...

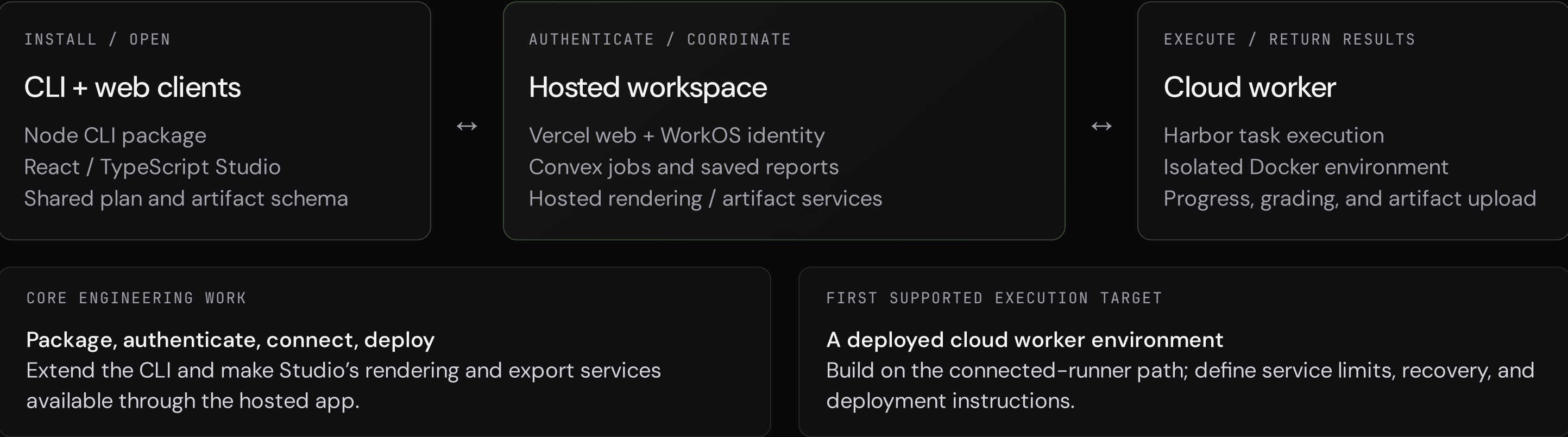
Then restore the original run’s status, retain one execution, and save one final report.

Done = acceptance checks + peer review + documentation + verification against the target deployment.

BUILD ON THE EXISTING CLI, HOSTED REPORTS, AND RUNNER

One platform. Two entry points.

The CLI and browser use the same evaluation plan, job identity, and result contract.



Evaluate managed sandbox automation after the first cloud path; keep compute replaceable behind the worker boundary.

SEMESTER MILESTONES FROM A WORKING FOUNDATION

Ship the path from install to cloud report.

FIRST RELEASE CYCLE	SECOND RELEASE CYCLE	THIRD RELEASE CYCLE	FINAL RELEASE CYCLE
Expand the CLI Account connection, shared run plans, submission commands, and packaged installation. Exit: install → sign in → run ID.	Complete the cloud app Deploy the full editor, report persistence, and hosted image and thread exports. Exit: browser-only edit → export.	Connect execution Deploy a cloud worker; connect queue, progress, cancel, recovery, and result sync. Exit: fresh remote run → saved report.	Pilot & release Test with new users, resolve workflow gaps, and publish the CLI and deployment guide. Exit: usable CLI + live cloud release.

Adoption Target: 4 of 5 new users install the CLI and submit a cloud run without cloning the repository.	Cloud completeness Run on a remote worker; open the report on a second device and export without a local server.	Execution reliability Browser closure and reconnect preserve run identity; cancellation and result upload pass acceptance checks.
--	--	---

Release evidence: a published package, a deployed application, a fresh cloud evaluation, and documented acceptance results.

THE SEMESTER PROMISE

Install the CLI.
Run in the cloud.
Publish the evidence.

Build on a completed evaluation and published results to make Heval available to developers and teams beyond the author’s machine.

Matthew Feroz

SSW 695-A · Software Engineering Capstone Studio
Fall 2026 · Prof. Rich Kempinski
github.com/MatthewFeroz/heval ↗

Feedback that will shape the release

What should the first CLI workflow feel like?
What makes a hosted evaluation easy to trust?
Which user group should pilot the release?